

Egypt University of Informatics
Faculty of Computing and Information Sciences
Computer Vision Course

CV-2: Crop Disease Detection

A Transfer Learning System for Plant Leaf Disease Classification

Project Information

Course: Computer Vision
Project Code: CV-2
Project Topic: Crop Disease Detection
Framework: PyTorch
Model: EfficientNet-B0 with Transfer Learning
Number of Classes: 38 crop and disease categories
GitHub Repository: <https://github.com/ziadabdelhafizz/crop-disease-detection>

Done By

Name	ID
Ziad Abdelhafiz	22-101236
Yehia Mahrous	22-101239
Seif Hegazy	22-101049

Under the Supervision of

Dr. Mona Soliman
Dr. Mennatullah Hassan

Academic Year 2025–2026

May 18, 2026

Contents

Abstract	2
1 Introduction	3
2 Problem Statement	4
3 Related Work	4
4 Dataset Description	5
4.1 Preprocessing	6
4.2 Data Augmentation	6
5 Methodology	7
5.1 Training Pipeline	7
5.2 Inference Pipeline	7
6 Model Architecture	8
6.1 Loss Function and Optimizer	8
6.2 Training Configuration	8
7 Explainability Using Grad-CAM	9
8 Experiments and Results	9
8.1 Training Curves	10
8.2 Final Test Results	11
9 Confusion Matrix and Error Analysis	11
10 Application Demo	13
10.1 Confidence-Based Reliability Message	13
10.2 Top-3 Prediction Output	14
11 GitHub Repository and Reproducibility	14
12 Limitations	15
13 Conclusion and Future Work	15
References	17

Abstract

Crop diseases are a major challenge in agriculture because they can reduce crop quality, productivity, and economic value. Early visual identification of plant diseases can support farmers and agricultural specialists in taking faster decisions before disease spread becomes severe. This project presents a computer vision system for crop disease detection using deep learning and transfer learning. The system classifies plant leaf images into 38 crop and disease categories using the PlantVillage-style New Plant Diseases Dataset.

The proposed system uses EfficientNet-B0 pretrained on ImageNet as the main convolutional neural network. The model was trained using PyTorch with data augmentation, normalization, label smoothing, the AdamW optimizer, learning-rate scheduling, and early stopping. The dataset was organized into training, validation, and testing splits containing 70,295 training images, 8,777 validation images, and 8,795 testing images. The final model achieved a test accuracy of 94.59%, weighted precision of 94.70%, weighted recall of 94.59%, and weighted F1-score of 94.58%.

In addition to classification, the project includes a command-line inference script and a Gradio web demo. The demo provides top-3 disease predictions, confidence-based reliability messages, and Grad-CAM explainability heatmaps. Grad-CAM was added to highlight the image regions that influenced the model prediction, making the system more transparent and suitable for academic discussion. The project is fully documented and available as a GitHub repository with reproducible training, evaluation, and inference instructions.

1 Introduction

Agriculture is one of the most important sectors for food production and economic stability. Crop diseases can negatively affect yield, quality, and profitability. Traditionally, disease identification depends on manual inspection by farmers or agricultural specialists. However, manual inspection can be slow, subjective, and difficult to scale, especially when diseases appear across large farming areas.

Computer vision provides an opportunity to automate the visual identification of crop diseases. Since many plant diseases produce visible symptoms on leaves, such as spots, discoloration, fungal patterns, yellowing, or blight regions, image classification models can learn to distinguish between healthy and diseased crops. Deep learning, especially convolutional neural networks, has become highly effective for image classification tasks because CNNs automatically learn spatial features from images.

This project focuses on crop disease detection from plant leaf images. The goal is to build a functional and accurate computer vision system that receives a plant leaf image and predicts the corresponding crop disease class. The system was designed as a university-level computer vision project with emphasis on clean code, reproducibility, evaluation, documentation, and practical demonstration.

The final system includes the following components:

- Dataset preparation and organization.
- Image preprocessing and augmentation.
- Transfer learning using EfficientNet-B0.
- Training, validation, and testing pipelines.
- Evaluation using accuracy, precision, recall, F1-score, and confusion matrix.
- Single-image inference script.
- Gradio web application for interactive prediction.
- Grad-CAM explainability heatmap.
- Confidence-based reliability message.
- GitHub repository with reproducible instructions.

2 Problem Statement

The main problem addressed in this project is the automatic classification of plant leaf images into crop disease categories. Given an input image of a plant leaf, the system must predict whether the plant is healthy or affected by a specific disease. This is formulated as a supervised multi-class image classification problem.

Let an input image be represented as x , and let the target class be represented as y , where y belongs to one of 38 possible classes. The objective is to learn a function:

$$f(x) = y$$

where f is a trained convolutional neural network that maps the input image to the correct crop disease label.

The project must satisfy the following practical requirements:

- Achieve strong classification accuracy on unseen test images.
- Support all 38 dataset classes.
- Provide reproducible training and evaluation code.
- Save and load the trained model.
- Provide inference on individual images.
- Include clear documentation and report results.
- Provide a simple application/demo interface.

3 Related Work

Crop disease detection using computer vision has become a common application of deep learning in agriculture. Earlier image processing methods depended on handcrafted features, color thresholding, texture descriptors, and traditional classifiers such as support vector machines or random forests. These methods required manual feature engineering and were often sensitive to lighting, background, and image quality.

Convolutional neural networks improved image classification by automatically learning hierarchical visual features. Lower layers typically learn edges and simple patterns, while deeper layers learn more complex structures such as leaf spots, disease textures, and crop-specific features. Transfer learning further improves performance by starting from models pretrained on large-scale datasets such as ImageNet. This is especially useful when the target dataset is smaller than general-purpose image datasets.

PlantVillage is a widely used dataset for plant disease recognition. It contains healthy and diseased plant leaf images grouped by crop species and disease type. Many research projects use PlantVillage-style datasets as a benchmark because they provide labeled data across multiple crops and diseases.

EfficientNet is a family of convolutional neural networks designed to balance accuracy and computational efficiency. EfficientNet-B0 is a lightweight member of the family and is suitable for university projects because it provides strong performance while remaining practical to train using Google Colab. In this project, EfficientNet-B0 was selected as the main model due to its balance of accuracy, speed, and simplicity.

Explainability methods such as Grad-CAM are also important in computer vision. Grad-CAM helps visualize which image regions contributed most to a CNN prediction. This is valuable in crop disease detection because it allows users to verify whether the model is focusing on disease symptoms rather than irrelevant backgrounds.

4 Dataset Description

The project uses the New Plant Diseases Dataset from Kaggle, which is based on PlantVillage-style crop disease classification. The dataset contains RGB images of plant leaves belonging to 38 classes. Each class represents either a healthy crop or a specific crop disease.

The dataset was organized into three splits: training, validation, and testing. The original dataset already contained training and validation folders. The validation set was divided into validation and test subsets to allow final unbiased testing.

Table 1: Dataset split used in the project.

Split	Number of Images
Training	70,295
Validation	8,777
Testing	8,795

The total number of classes is 38. Example classes include Apple Apple Scab, Corn Common Rust, Grape Black Rot, Potato Early Blight, Tomato Late Blight, and healthy crop classes.

	split	num_classes	num_images
0	train	38	70295
1	val	38	8777
2	test	38	8795

Figure 1: Dataset split summary used for training, validation, and testing.

4.1 Preprocessing

All images were resized to 224 by 224 pixels to match the input requirements of EfficientNet-B0. Images were converted to tensors and normalized using ImageNet mean and standard deviation values:

$$mean = [0.485, 0.456, 0.406]$$

$$std = [0.229, 0.224, 0.225]$$

This normalization is appropriate because the model uses ImageNet pretrained weights.

4.2 Data Augmentation

Data augmentation was applied to the training set to improve generalization. The following transformations were used:

- Random resized crop.
- Random horizontal flip.
- Random vertical flip.
- Random rotation.
- Color jitter for brightness, contrast, saturation, and hue.

Validation and test images were not randomly augmented. They were resized, center-cropped, converted to tensors, and normalized.

5 Methodology

The project follows a standard supervised deep learning workflow. First, the dataset was downloaded and organized into training, validation, and test folders. Then, image transformations were defined for training and evaluation. A pretrained EfficientNet-B0 model was loaded, and its final classifier layer was replaced with a new classifier matching the 38 target classes.

The model was trained using transfer learning. During the baseline training stage, the pretrained feature extractor was frozen and only the classification head was trained. This approach reduces training time and allows the model to reuse general image features learned from ImageNet.

5.1 Training Pipeline

The training pipeline includes the following steps:

1. Load the dataset using PyTorch ImageFolder.
2. Apply training and evaluation transforms.
3. Create DataLoaders for training, validation, and testing.
4. Load EfficientNet-B0 with pretrained ImageNet weights.
5. Replace the final classifier with a 38-class output layer.
6. Train the model using CrossEntropyLoss.
7. Monitor validation accuracy after each epoch.
8. Save the best model based on validation accuracy.
9. Generate loss and accuracy curves.

5.2 Inference Pipeline

During inference, a user-provided leaf image is resized, center-cropped, normalized, and passed through the trained model. The model outputs probability scores for all 38 classes using the softmax function.

Instead of displaying only the highest-probability class, the system displays the top-3 predicted classes. This is useful because some crop diseases are visually similar. For example, different tomato diseases may appear as spotted or discolored regions, while maize diseases may share elongated lesion patterns. Showing the top-3 predictions provides more information when the model is uncertain.

6 Model Architecture

The final model uses EfficientNet-B0 as the backbone. EfficientNet-B0 is a convolutional neural network designed to achieve a good balance between accuracy and efficiency. It uses compound scaling to balance network depth, width, and resolution.

In this project, EfficientNet-B0 was initialized with ImageNet pretrained weights. The original classifier layer was replaced with a new classification head:

$$\text{Dropout}(0.3) \rightarrow \text{Linear}(\text{in_features}, 38)$$

The dropout layer helps reduce overfitting, while the final linear layer maps the extracted image features to the 38 crop disease classes.

6.1 Loss Function and Optimizer

The loss function used was CrossEntropyLoss with label smoothing. Label smoothing helps prevent the model from becoming overconfident by slightly softening the target labels during training.

The optimizer used was AdamW. AdamW is suitable for deep learning because it combines adaptive learning rates with decoupled weight decay. A learning-rate scheduler was also used to reduce the learning rate when validation accuracy stopped improving.

6.2 Training Configuration

Table 2: Training configuration.

Parameter	Value
Model	EfficientNet-B0
Input size	224×224
Batch size	32
Optimizer	AdamW
Learning rate	0.001
Weight decay	0.0001
Loss function	CrossEntropyLoss
Label smoothing	0.05
Epochs	5
Early stopping patience	3
Framework	PyTorch
Training environment	Google Colab GPU

7 Explainability Using Grad-CAM

Although classification accuracy is important, a crop disease detection system should also provide some level of interpretability. In real agricultural applications, users should not only receive a predicted disease label, but should also understand which visual regions influenced the model’s decision. For this reason, Gradient-weighted Class Activation Mapping, known as Grad-CAM, was added as an explainability component.

Grad-CAM produces a heatmap that highlights the regions of the input image that contributed most strongly to the predicted class. In this project, Grad-CAM was applied to the final convolutional feature block of the EfficientNet-B0 model. The resulting heatmap was resized to match the input image and then overlaid on top of the original leaf image.

This is useful for crop disease detection because many diseases appear as localized symptoms such as spots, discoloration, edge damage, blight regions, or infected patches. If the highlighted regions overlap with visible disease symptoms, this provides additional confidence that the model is using meaningful visual evidence.

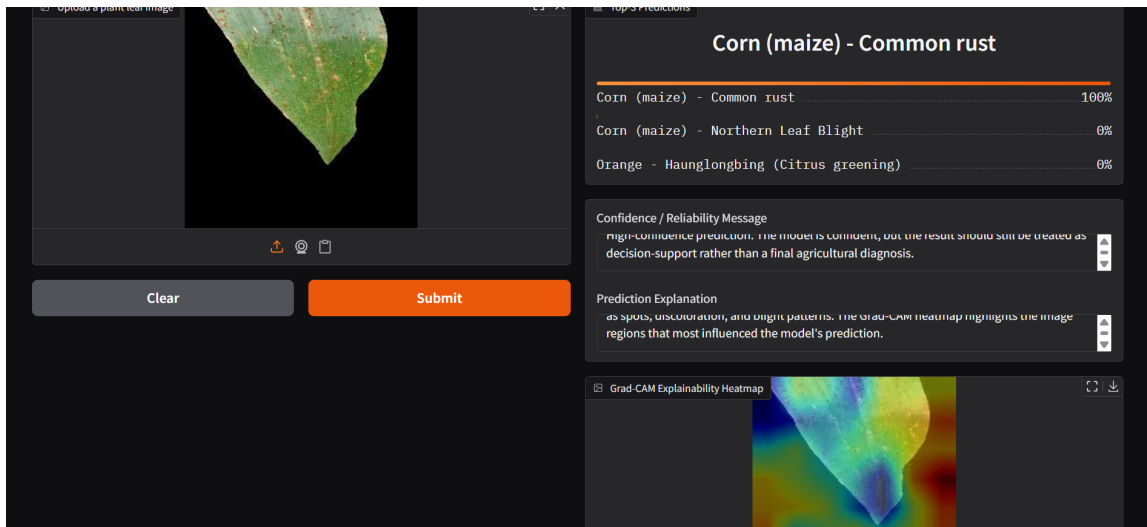


Figure 2: Updated Gradio demo showing top-3 predictions, confidence warning, prediction explanation, and Grad-CAM heatmap.

The Grad-CAM feature improves the transparency of the model and makes the demo more useful during discussion. It also helps show that the project goes beyond simple classification by adding an explainability layer.

8 Experiments and Results

The model was trained for 5 epochs using the training set and evaluated after each epoch using the validation set. The best validation accuracy was achieved at the final epoch.

The final model was then evaluated on the independent test set.

8.1 Training Curves

The loss curve shows that training and validation loss decreased over time, indicating that the model learned useful features from the data.

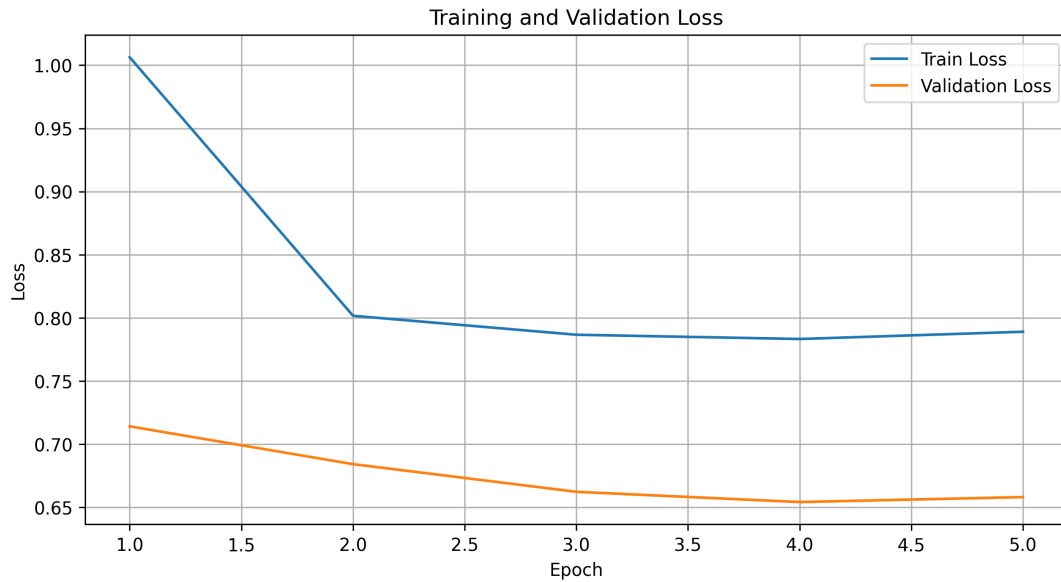


Figure 3: Training and validation loss curves.

The accuracy curve shows that validation accuracy improved steadily and reached 94.49%. The training accuracy was lower than validation accuracy because data augmentation, dropout, and label smoothing made the training process more difficult. This is acceptable and does not indicate a problem.

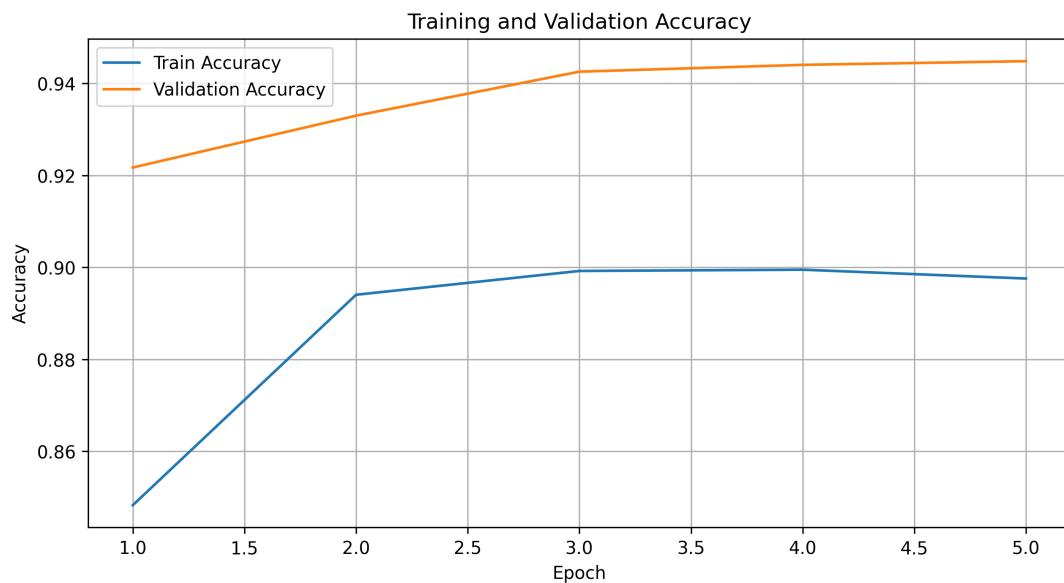


Figure 4: Training and validation accuracy curves.

8.2 Final Test Results

The final test results are shown in Table 3. These results were obtained on 8,795 test images.

Table 3: Final test results.

Metric	Result
Test Accuracy	94.59%
Weighted Precision	94.70%
Weighted Recall	94.59%
Weighted F1-score	94.58%
Test Loss	0.2401
Number of Test Images	8,795
Number of Classes	38

The weighted precision, recall, and F1-score are close to the overall test accuracy, which indicates balanced performance across the dataset. The result is strong for a university-level computer vision project and demonstrates that transfer learning with EfficientNet-B0 is effective for crop disease classification.

9 Confusion Matrix and Error Analysis

A confusion matrix was generated to understand which classes were correctly classified and which classes were confused. The normalized confusion matrix is shown in Figure 5.

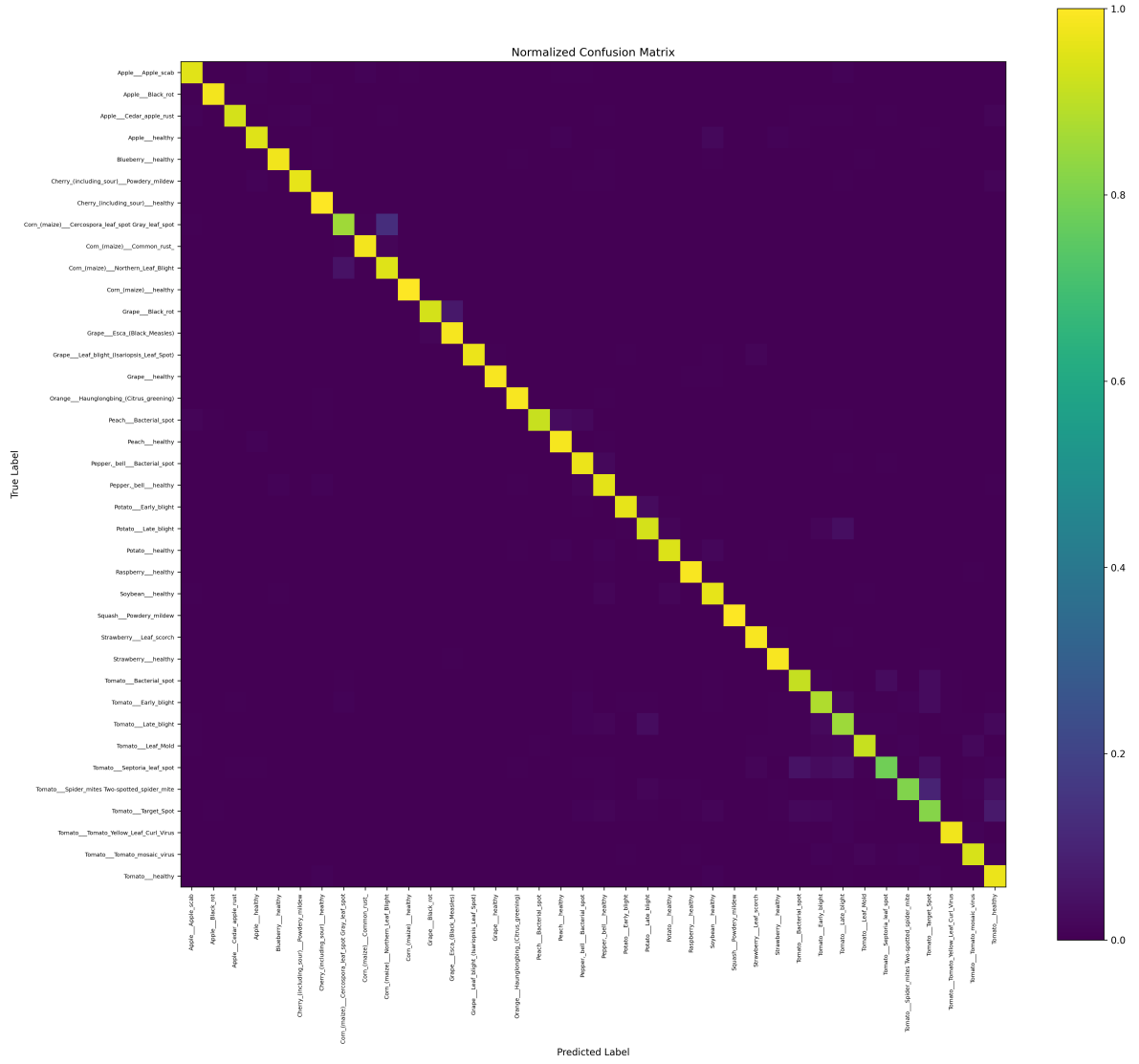


Figure 5: Normalized confusion matrix for the 38-class crop disease classification task.

Most classes achieved strong classification performance. However, some visually similar diseases were confused. The most common confusion cases are shown in Table 4.

Table 4: Top confusion cases from the test set.

True Class	Predicted Class	Count
Corn Cercospora leaf spot / Gray leaf spot	Corn Northern Leaf Blight	27
Tomato Spider mites	Tomato Target Spot	22
Tomato Target Spot	Tomato healthy	16
Grape Black rot	Grape Esca / Black Measles	15
Corn Northern Leaf Blight	Corn Cercospora leaf spot / Gray leaf spot	12
Tomato Septoria leaf spot	Tomato Bacterial spot	11
Potato Late blight	Tomato Late blight	9
Tomato Septoria leaf spot	Tomato Late blight	9
Tomato Late blight	Potato Late blight	8
Tomato Septoria leaf spot	Tomato Target Spot	8

The highest confusion occurred between Corn Cercospora leaf spot / Gray leaf spot and Corn Northern Leaf Blight. This is reasonable because both diseases affect maize leaves and can produce visually similar lesions. Tomato diseases also appeared frequently in the confusion list because multiple tomato leaf diseases produce spots, yellowing, or irregular discoloration. Some confusion between potato and tomato late blight may occur because both disease classes contain blight-like visual symptoms.

This error analysis shows that the model performs well overall but still struggles with diseases that share similar visual patterns. These results support the use of top-3 predictions because the correct class may sometimes be close to the top prediction even when the top-1 class is wrong.

10 Application Demo

A Gradio-based web application was implemented to make the system interactive and easy to test. The user uploads a plant leaf image, and the trained EfficientNet-B0 model returns the predicted crop disease classes with confidence scores.

The final demo includes the following features:

- Uploading a plant leaf image.
- Returning top-3 predicted classes.
- Displaying confidence scores.
- Showing a confidence-based reliability message.
- Providing a short explanation of the prediction.
- Displaying a Grad-CAM heatmap.

10.1 Confidence-Based Reliability Message

The confidence warning system was added to make the demo more realistic. If the model confidence is high, the system states that the model is confident while still treating the result as decision-support. If confidence is medium or low, the system warns that the image may be ambiguous, unclear, or different from the training distribution.

This is important because real-world plant images may include complex backgrounds, shadows, lighting variation, overlapping leaves, or symptoms not present in the dataset. Therefore, the system should not be presented as a replacement for expert agricultural diagnosis.

10.2 Top-3 Prediction Output

The top-3 output provides more information than a single prediction. It is especially useful when disease symptoms are visually similar. For example, several tomato diseases produce leaf spots, and several blight diseases produce irregular discoloration. Showing the top-3 predictions helps users understand alternative possibilities and model uncertainty.

11 GitHub Repository and Reproducibility

The project was organized into a professional GitHub repository. The repository contains the source code, trained model, evaluation outputs, README file, requirements file, and application demo.

The repository structure includes:

- `src/dataset.py`: dataset loading and transformations.
- `src/model.py`: EfficientNet-B0 model definition.
- `src/train.py`: training pipeline.
- `src/evaluate.py`: test evaluation pipeline.
- `src/predict.py`: single-image inference script.
- `src/gradcam.py`: Grad-CAM explainability code.
- `app/gradio_app.py`: Gradio demo application.
- `models/best_model.pth`: trained model checkpoint.
- `outputs/`: training curves, confusion matrix, reports, and prediction outputs.
- `README.md`: installation, dataset, training, inference, and demo instructions.

The GitHub repository is available at:

The repository allows the project to be reproduced by installing dependencies, downloading the dataset, organizing the folders, training the model, evaluating it, and running the Gradio demo.

12 Limitations

Although the final model achieved strong test performance, the system has some limitations.

First, the dataset mainly contains controlled leaf images with clear views of leaves. Real agricultural environments are more complex. Images captured in the field may contain different lighting conditions, complex backgrounds, soil, multiple leaves, motion blur, or partial occlusion.

Second, the model performs image-level classification only. It predicts the disease class but does not segment the exact diseased region. Grad-CAM provides a visual explanation, but it is not a precise segmentation mask.

Third, the model only supports the 38 classes present in the dataset. If the uploaded image belongs to a crop or disease outside the dataset, the model will still force the prediction into one of the known classes. This is why the confidence warning is important.

Fourth, the system is not a final agricultural diagnosis tool. It should be considered a decision-support system that can help guide users but not replace expert verification.

13 Conclusion and Future Work

This project successfully developed a crop disease detection system using computer vision and deep learning. The system classifies plant leaf images into 38 crop and disease categories using EfficientNet-B0 transfer learning. The dataset was prepared, trained, evaluated, and documented using a reproducible pipeline.

The final model achieved strong performance with 94.59% test accuracy, 94.70% weighted precision, 94.59% weighted recall, and 94.58% weighted F1-score. The evaluation included training curves, confusion matrix, classification report, top confusion analysis, and sample predictions.

The final system also includes a Gradio demo that supports top-3 predictions, confidence-based reliability messages, prediction explanation, and Grad-CAM heatmap visualization. These additions improve the practicality, transparency, and presentation quality of the project.

Future work can include:

- Training on more real-world field images.
- Adding leaf segmentation before classification.
- Comparing multiple architectures such as MobileNetV3, ResNet50, and Vision Transformers.
- Fine-tuning the full EfficientNet backbone.

- Adding out-of-distribution detection for unknown crops or diseases.
- Deploying the model as a mobile or web application.
- Collecting local crop disease images for better real-world robustness.

Overall, the project meets the requirements of a functional, accurate, documented, and reproducible computer vision system for crop disease detection.

References

- [1] Hughes, D. P., and Salathé, M. *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. PlantVillage Dataset.
- [2] Tan, M., and Le, Q. V. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. International Conference on Machine Learning, 2019.
- [3] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. International Journal of Computer Vision, 2020.
- [4] Paszke, A. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems, 2019.
- [5] Deng, J. et al. *ImageNet: A Large-Scale Hierarchical Image Database*. IEEE Conference on Computer Vision and Pattern Recognition, 2009.